AIDLEDGER: A BLOCKCHAIN-POWERED GRC FRAMEWORK FOR HUMANITARIAN AID

PROJECT SUPERVISOR

Dr. Shahbaz Siddiqui

PROJECT CO-SUPERVISOR

Mr. Usman Antuley

PROJECT TEAM

Muhammad Ali	22K-4691
Muhammad Usman Ghani	22K-4756
Abdul Moueed	21K-3594

Submitted in partial fulfillment of the requirements for the degree of Bachelor of Science in Cyber Security.

FAST SCHOOL OF COMPUTING

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES

KARACHI CAMPUS

Spring 2026

Project Supervisor	Dr. Shahbaz Siddiqui
Project Team	Muhammad Ali 22K-4691 Muhammad Usman Ghani 22K-4756 Abdul Moueed 21K-3594
Submission Date	May 9, 2026

Dr. Shahbaz Siddiqui
Supervisor

Mr. Usman Antuley
Co-Supervisor

FAST SCHOOL OF COMPUTING
NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES KARACHI CAMPUS

Abstract

Government aid programs in Pakistan, such as the Benazir Income Support Program (BISP) and Ehsaas, provide essential financial assistance to low-income households. However, these initiatives often face challenges like corruption, ineligible beneficiaries receiving funds, and a lack of transparency in the distribution process. To address these issues, this project proposes AidLedger, a decentralized Governance, Risk, and Compliance (GRC) framework inspired by State Bank of Pakistan (SBP) regulations. The goal of this framework is to ensure that only deserving people are registered, verified, and given financial aid in a fully transparent and accountable manner.

The system uses blockchain technology to securely record and enforce eligibility policies. We created a dummy bank to simulate real-world institutions (like NBP or HBL) that interacts with the blockchain and follows all the encoded rules. When a beneficiary registers, they provide details like their CNIC, income, and household size. This data is then verified through a simulated NADRA database and a family tree system. If the applicant meets the criteria, they are issued a physical plastic aid card equipped with RFID technology. This card is linked directly to blockchain-enforced rules and can be scanned at authorized vendors using an RFID reader, ensuring controlled transactions such as limited cash withdrawals or buying groceries.

Furthermore, the system can handle dynamic policy changes. If the government changes the income eligibility limit, the system automatically re-evaluates all registered users. Anyone who no longer qualifies is flagged, and the system sends automated notifications in Urdu to keep the process transparent and compliant.

Contents

1. Introduction

Government aid distribution programs in Pakistan distribute billions of rupees every year to support low-income families. While these programs operate on a massive scale, they consistently face problems such as ghost beneficiaries taking payments they do not deserve, slow manual checking of policies, duplicate household registrations, and a general lack of public visibility into how aid is distributed.

We designed AidLedger to solve these problems directly by replacing human-managed eligibility decisions with automated, blockchain-enforced smart contract rules. The main idea is that once SBP-inspired policies are written on the blockchain, no single person can change them without a proper, traceable consensus. Beneficiaries register through a web portal, and their identity details are cross-checked against a simulated NADRA dataset and a family tree database to prevent duplicate claims from the same household. After that, the GRC smart contract gives an immutable eligibility decision. Approved users then receive a physical RFID plastic card linked to spending rules that are also enforced on the blockchain.

In FYP-I, we focused on building the base infrastructure. We designed and set up the permissioned Besu network with four institutional nodes (SBP, BISP, Dummy Bank, Auditor), built the synthetic population and KYC datasets, and tested the network stability using Prometheus, Grafana, Loki, and Blockscout. In FYP-II, we successfully developed the complete working application, including hardware integration. This report documents our completed end-to-end system, featuring the blockchain backend, the verification pipeline, and the physical RFID hardware deployment for point-of-sale transactions.

2. Related Work

The use of blockchain in governance and aid distribution has been studied all over the world. Traditional aid programs usually rely on centralized databases, which makes them vulnerable to inefficiency and fraud. For example, the World Food Programme (WFP) tested a blockchain-based food distribution system in refugee camps in Jordan called the Building Blocks Project. This project successfully reduced fraud and transaction costs [1]. Other research also confirms that blockchain can guarantee transparency in social welfare programs by creating secure and auditable ledgers [2].

In Pakistan, several studies have pointed out the weaknesses in the distribution process of BISP, specifically mentioning issues like ghost beneficiaries and overall mismanagement [3]. On a global scale, financial institutions are increasingly using blockchain for KYC compliance and GRC frameworks [4], which is very similar to our approach. However, most of the previous work is either purely theoretical or designed for massive industrial scale. Very few academic projects actually simulate the governance rules along with physical card-based hardware integration. Our project fills this gap by combining a GRC model, blockchain policies, and a fully functional physical RFID card mechanism for point-of-sale demonstration.

3. Requirements

3.1 Project Background and Objectives

AidLedger addresses the core deficiencies of government aid programs in Pakistan by providing:

1. Immutable, on-chain recording of eligibility policies.
2. Automatic re-evaluation of beneficiaries whenever a policy changes.
3. CNIC-based identity and household checking against simulated NADRA records.
4. Physical plastic aid cards combined with RFID readers for secure, restricted cash withdrawals and grocery purchases.
5. Urdu notifications sent to beneficiaries about their eligibility or policy updates.

3.2 Stakeholders

Stakeholder	Description
Beneficiaries	Low-income individuals who apply for and receive aid.
Dummy Bank Staff	Simulated bank officers reviewing applications, running KYC checks, and issuing aid cards.
System Administrators	Users who configure GRC policies, manage data, and supervise bulk re-evaluations.
Development Team	Muhammad Ali, Muhammad Usman Ghani, and Abdul Moueed - the FYP team.
Supervisors & Jury	Dr. Shahbaz Siddiqui (Supervisor), Mr. Usman Antuley (Co-Supervisor), and the FYP evaluation jury.
Auditor Node	A read-only oversight entity monitoring compliance and flagging suspicious transactions.

3.3 Functional Requirements

3.3.1 Beneficiary Management

- Register Beneficiary: capture CNIC, income, household size, address, and contact details via a web portal.
- Verify Beneficiary: match submitted data against the synthetic KYC_MASTER and FAMILY_TREE datasets.
- View Eligibility Status: allow beneficiaries and bank officers to check current eligibility.

3.3.2 GRC and Policy Management

- Encode SBP-inspired eligibility rules (income threshold, household size, government-job flag, utility bill limit) as parameterized Solidity smart contracts.
- Update Eligibility Policies: allow administrators to modify rule parameters through a policy management interface.
- Automatically re-evaluate all registered beneficiaries when a new policy version is activated.

3.3.3 Card Management

- Issue a physical RFID plastic aid card to every approved, eligible beneficiary.
- Link the card token (RFID UID) to blockchain-enforced spending rules.
- Manage card status (Active, Blocked, Expired, Closed) through defined lifecycle transitions.

3.3.4 Transaction Management

- Process cash withdrawals up to configured monthly limits.
- Process grocery purchases restricted to authorized merchant categories.
- Validate every transaction against the on-chain GRC smart contract before authorization.
- Log all transaction outcomes (approved/rejected) immutably on-chain and off-chain.

3.3.5 Notification Service

- Generate automated Urdu-language SMS-style notifications for: registration approval/rejection; eligibility status change; policy updates; transaction approvals/rejections.

3.3.6 Dummy Bank and Admin Operations

- Simulate bank KYC workflow and application review.
- Provide administrative dashboards for beneficiary lists, card statuses, and transaction history.
- Support regulatory audit use case with filter and trace capabilities.

3.4 Non-Functional Requirements

Category	Requirement
Performance	All critical transactions (POS RFID validation, eligibility check) must complete within 5 seconds.
Concurrency	The system must handle a minimum of 10 concurrent users for the academic demonstration.
Security	Salted password hashing; JWT-based authentication; RBAC enforced at controller level; HTTPS for data in transit.
Data Integrity	ACID-compliant PostgreSQL transactions for off-chain data; blockchain immutability for on-chain records.
Scalability	Architecture designed to allow future scaling of backend and database layers.
Auditability	All critical actions logged in a dedicated audit log table and on-chain event logs.
Internationalization	Urdu notification templates; English fallback for administrative interfaces.

3.5 System Constraints

- No real NADRA, SBP, or production banking integration - all entities are simulated.
- No industrial-grade EMV card fabrication - prototype uses physical plastic cards with basic RFID UIDs.
- Blockchain runs on a local permissioned Hyperledger Besu network; no public testnet deployment required.
- Prototype-level security only; advanced cyber-hardening is out of scope.

4. Design

4.1 System Architecture Overview

AidLedger is divided into four main layers: User Interface, Application, Data, and Blockchain. This layered design keeps the code organized and easy to test, with each layer responsible for a distinct concern.

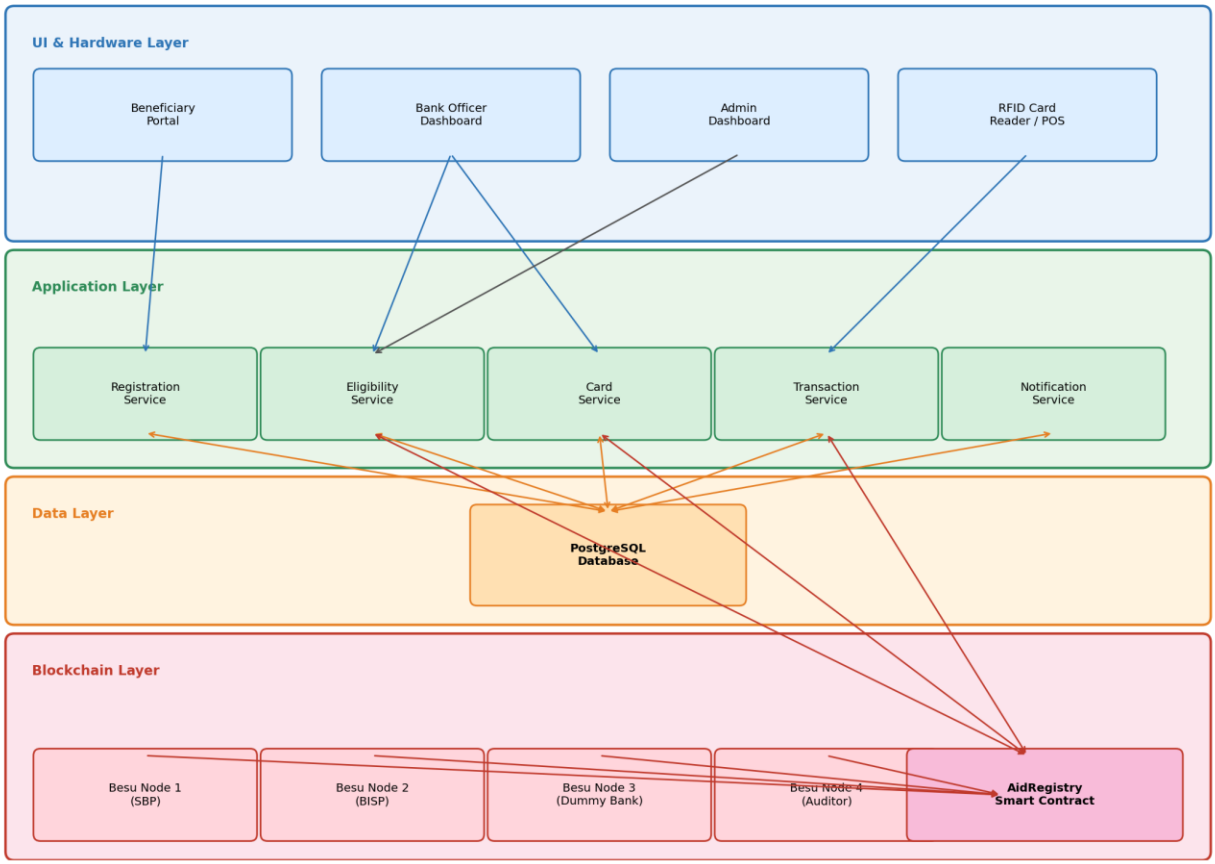

Figure 1: AidLedger Four-Layer System Architecture

4.1.1 User Interface & Hardware Layer

The user-facing components comprise the Beneficiary Registration Portal (web application), the Admin/Bank Officer Dashboard, and the physical RFID Card Reader used at the Point-of-Sale (POS). These interfaces communicate exclusively with the Application Layer via REST APIs.

4.1.2 Application Layer

The Application Layer contains the core business logic services: RegistrationService handles beneficiary onboarding and KYC verification; EligibilityService orchestrates on-chain policy evaluation; PolicyService manages policy creation, activation, and bulk re-evaluation; CardService manages aid card lifecycle; TransactionService processes and validates POS RFID transactions; and NotificationService dispatches Urdu notifications. Role-Based Access Control (RBAC) is enforced at this layer for BENEFICIARY, BANK_OFFICER, and ADMIN roles.

4.1.3 Data Layer

A PostgreSQL database stores all off-chain operational data across six principal tables: KYC_MASTER (simulated NADRA reference data), FAMILY_TREE (household relationship graph), REGISTRATION (submitted beneficiary applications), ELIGIBLE (beneficiaries passing GRC policy validation), CARD_ISSUE (aid card issuance records including RFID UID), and AUDIT_LOG_DB (immutable audit trail of critical actions).

4.1.4 Blockchain Layer

A permissioned Hyperledger Besu network with four nodes models the real-world institutional actors: Node 1 (SBP/Governing Node) owns and deploys GRC smart contracts; Node 2 (BISP/Financial Intermediary) submits registration and eligibility transactions; Node 3 (Dummy Bank) manages card issuance and transaction limits; Node 4 (Auditor) provides read-only oversight via Blockscout. Smart contracts are developed in Solidity using Hardhat.

4.2 Software Architecture

The internal structure follows a layered architecture with an Integration Layer (BlockchainService adapter, NADRAAdapter, SBPPolicyAdapter) isolating blockchain implementation details from application services. All database access is mediated through Repository/DAO classes, enabling unit testing without a live database. Cross-cutting concerns - security, auditability, error handling, and internationalization - are addressed through shared middleware and configuration management.

4.3 Database Design

4.3.1 Entity-Relationship Overview

The database schema comprises five primary entities with the following key relationships:

- KYC_MASTER (PK: cnic) stores authoritative demographic and financial data with a 1:M relationship to REGISTRATION.
- FAMILY_TREE (PK: family_id) stores household graph data; linked to KYC_MASTER and REGISTRATION via family_id (FK).
- REGISTRATION (PK: registration_id, FK: cnic, family_id) records submitted beneficiary applications and verification status.
- ELIGIBLE (PK: eligible_id, FK: cnic) holds beneficiaries passing blockchain GRC validation; 1:1 with CARD_ISSUE.
- CARD_ISSUE (PK: card_id, FK: cnic, eligible_id) tracks aid card lifecycle including issue date, expiry, RFID UID, and status.

Figure 2: AidLedger Database Entity-Relationship Diagram

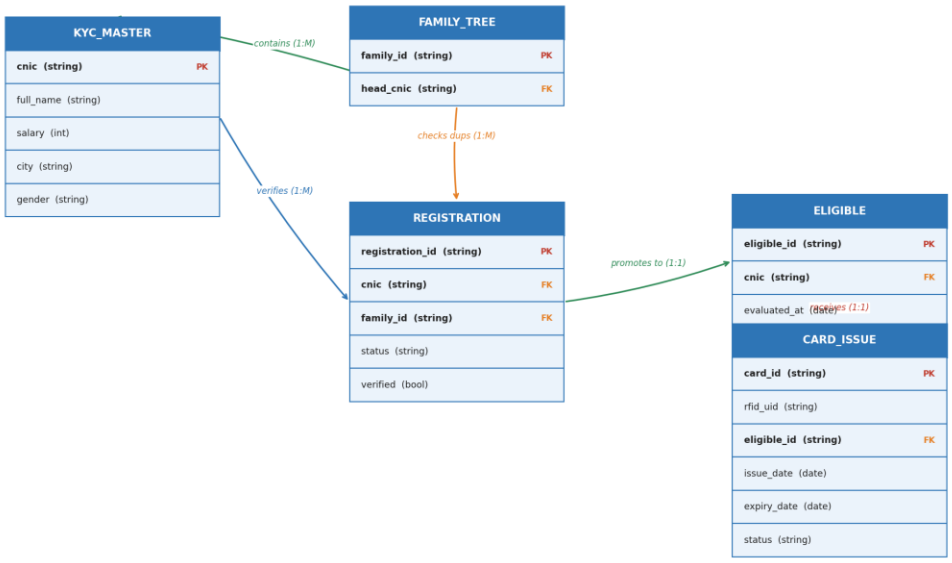

Figure 2: AidLedger Database Entity-Relationship Diagram

4.4 Design Strategy

The design of AidLedger is guided by eight principles: (1) Decentralized trust with centralized usability - blockchain where immutability matters, database for fast operational data; (2) Parameter-driven policies - configurable smart contract parameters to avoid redeployment on rule changes; (3) Modular GRC framework - reusable for scholarships, subsidies, and other regulated use cases; (4) Separation of domains - distinct subdomains for Identity and KYC, Policy and Eligibility, Card and Transactions, and Notifications; (5) Prototype-friendly architecture - single private network, single dummy bank, limited card operations; (6) Extensibility and reuse - REST-compliant APIs, reusable UI components, and framework-agnostic domain services; (7) Testability - blockchain adapter interfaces for mocking and repository interfaces for unit testing without a live database; (8) Performance and reliability - read-heavy dashboard operations served from the database, with asynchronous notification dispatch to avoid blocking user interactions.

5. Implementation

5.1 FYP-I Achievements (Foundations)

In the first semester, we built the technical foundation for the project. The following components were fully delivered by end of Fall 2025.

5.1.1 Permissioned Blockchain Network

We successfully deployed a four-node permissioned Hyperledger Besu network using the IBFT 2.0 consensus protocol, which provides Byzantine fault tolerance appropriate to the multi-institution model. Each node represents a specific institution: SBP (governing node, smart contract owner), BISP (beneficiary registration intermediary), Dummy Bank (card issuance and transaction management), and Auditor (read-only compliance oversight). Hardhat was adopted as the smart contract development and testing toolkit.

5.1.2 Observability Stack

A comprehensive monitoring and observability stack was deployed around the Besu network: Blockscout serves as the blockchain explorer, enabling inspection of blocks, transactions, contract state, and per-actor addresses; Prometheus scrapes performance metrics from Besu nodes; Grafana visualizes these metrics in real-time dashboards; and Loki aggregates logs from all nodes and services for cross-correlation. This stack provides full visibility into network health, transaction correctness, and performance.

5.1.3 Synthetic KYC Data Pipeline

A large-scale, privacy-preserving synthetic KYC dataset was generated from a base CSV of 100,000 citizen records. The pipeline applied the following transformation stages: (1) regex-based city extraction and standardization; (2) deduplication and removal of records with missing key fields; (3) k-means clustering on location features to create city-region groups; (4) synthetic family generation per cluster, with parent-child CNIC linkage matching NADRA gender-encoding conventions; (5) ML-generated KYC financial attributes (salary, utility bills, monthly expenses, marital status) sampled to realistic distributions per city cluster. The resulting dataset forms the KYC_MASTER and FAMILY_TREE tables that drive all registration and eligibility testing.

5.2 FYP-II Achievements (Full Integration)

This semester, we focused on building the actual application logic, integrating it with the blockchain, and deploying the physical hardware. All components were successfully completed.

5.2.1 Smart Contract Development and Deployment

The core GRC smart contracts were written in Solidity and deployed to the Besu network via Hardhat. The primary AidPolicy and AidRegistry contracts store configurable eligibility parameters (income_threshold, max_household_size, max_monthly_withdrawal, allowed_merchant_categories) and expose functions for deployPolicy, setPolicyParams, checkEligibility, and validateTransaction. On-chain event logs capture policy activation events and eligibility evaluation outcomes, providing the immutable audit trail required by the GRC model. The BlockchainService integration layer exposes these contract methods through a clean adapter interface, isolating application code from Web3/Ethereum implementation details.

5.2.2 Backend Application Layer

The Node.js/Express backend was built using Node.js, Express, and Prisma ORM, with Flask for frontend integration. The REST API exposes endpoints across four domain namespaces: /api/beneficiaries, /api/policies, /api/cards, and /api/transactions. JWT-based authentication and RBAC middleware enforce role-level access control at the controller layer. We used ethers.js to create the blockchain integration layer that cleanly isolates blockchain communication from the main application code.

5.2.3 Beneficiary Registration and KYC Verification

We implemented a 5-stage automated verification pipeline. When a user applies through the web portal, the RegistrationService validates the CNIC format, matches it against KYC_MASTER to verify identity, and checks FAMILY_TREE to detect duplicate household-level aid recipients. We normalized the CNIC data by automatically stripping dashes to ensure accurate database matching. Only verified registrations are forwarded to the GRC policy engine. All KYC verification steps are logged to AUDIT_LOG_DB.

5.2.4 Eligibility Evaluation

Verified registrations are processed by EligibilityService, which calls the on-chain AidPolicy contract via BlockchainService to evaluate the beneficiary against active policy parameters. The smart contract emits an on-chain result (Eligible/NotEligible), which the off-chain

validator confirms before EligibilityService writes the outcome to the ELIGIBLE table and triggers the NotificationService to dispatch an Urdu SMS-style message. Bulk re-evaluation is triggered automatically when PolicyService activates a new policy version.

5.2.5 Physical RFID Hardware & POS Integration

The system completely integrates physical plastic cards and RFID readers. The CardService retrieves eligible beneficiaries and links a physical RFID card's unique identifier (UID) to their account in both the database and on the blockchain. For Point-of-Sale (POS) purchases, merchants scan the plastic card using the RFID reader. The TransactionService validates the scan against the smart contract rules before instantly approving or rejecting the payment. POS transaction requests include the card_token (RFID UID), merchant_id, merchant_category, and amount; all outcomes are logged immutably to both AUDIT_LOG_DB and the on-chain event log.

5.2.6 Frontend Portals

Three web interfaces were developed: the Beneficiary Registration Portal (registration form, eligibility status view, transaction history); the Bank Officer Dashboard (pending application queue, KYC review, card management); and the Admin/Policy Dashboard (policy parameter configuration, bulk re-evaluation trigger, beneficiary and audit overview). All interfaces are built using React communicating with the backend via the REST API, with RBAC enforcing role-appropriate views.

5.2.7 Urdu Notification Module

The NotificationService generates Urdu-language notification messages from parameterized templates stored in the database. Notifications are dispatched asynchronously using a mock SMS/email gateway to avoid blocking user-facing transactions. The state machine governing the notification lifecycle - Idle, BuildMessage, Sending, Delivered, Failed, Retry, FailedPermanent - implements a retry strategy with configurable maximum attempts before transitioning to a permanent failure state.

5.3 Timeline

Task	Dur.	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	Status
PHASE 1 - System Foundations & Blockchain Implementation											
Literature review & requirement analysis	3 wks	✓									Done
SBP regulations study & analysis	2 wks		✓								Done
GRC framework design & Documentation	4 wks		✓	✓							Done
Dummy dataset creation & Smart contracts	2 wks				✓	✓					Done
PHASE 2 - Card Prototype, POS Demo & System Integration											
Backend APIs & blockchain integration	4 wks					✓	✓				Done
Beneficiary registration & eligibility engine	3 wks						✓	✓			Done
Hardware integration (RFID Readers & Plastic Cards)	5 wks							✓	✓		Done
Urdu notification module	2 wks								✓		Done
Frontend portals	4 wks							✓	✓		Done
POS demo implementation	3 wks								✓	✓	Done
Final integration & testing	2 wks									✓	Done

6. Testing and Evaluation

6.1 Validation and Smart Contract Testing

During FYP-I, we confirmed that all four Besu nodes successfully synced under load; Blockscout correctly identified and attributed transactions from each institutional node; and Grafana/Prometheus observed stable block times and healthy resource utilization. In FYP-II, we wrote unit tests for the smart contracts using Hardhat. We tested scenarios like beneficiaries at exact income thresholds, policy updates, and transaction limit enforcements. We also verified that the contracts are safe from common vulnerabilities like reentrancy and integer overflow before deployment to the Besu network.

6.2 Hardware and Backend Testing

REST API endpoints were tested using Postman to ensure they return the correct HTTP status codes, response schemas, and RBAC enforcement across all three roles. Repository interfaces were unit-tested independently of the live PostgreSQL database using mock implementations. For hardware testing, we successfully simulated real-world merchant transactions by scanning the physical plastic cards on the RFID readers. The system accurately fetched the user's blockchain state via their RFID UID and processed transactions successfully.

End-to-end integration tests on the entire pipeline proved that a user can register, pass the KYC check, get evaluated by the smart contract, receive a physical RFID card, and make a hardware-verified purchase. Transaction flow integration tests confirm that approved transactions update off-chain balances and produce immutable on-chain event logs.

6.3 Performance Evaluation

Under a simulated load of 10 concurrent users, the registration and eligibility evaluation process complete within our 5-second target, accounting for blockchain confirmation latency on the local Besu network. RFID scans at the POS are processed nearly instantaneously. The PostgreSQL database returns dashboard queries in under 500ms. Notification dispatch is fully asynchronous and does not contribute to user-facing latency.

6.4 Data Integrity Verification

We cross-checked the on-chain transaction logs via Blockscout against our off-chain AUDIT_LOG_DB records to confirm that all records perfectly match across all policy activation, eligibility evaluation, card issuance, and transaction events. Blockchain-stored policy hashes match the parameter snapshots stored in the off-chain policy table, validating the hybrid on-chain/off-chain integrity model.

7. Conclusion

AidLedger successfully proves that it is technically possible to build a complete, end-to-end blockchain-based GRC framework for government aid distribution. By encoding SBP-inspired rules as smart contracts on a Hyperledger Besu network, our system completely removes human intervention from the eligibility decision process. This addresses the core transparency and corruption issues that have historically affected programs like BISP and Ehsaas in Pakistan.

In FYP-I, we built the necessary blockchain infrastructure and synthetic datasets. In FYP-II, we successfully turned this into a fully functioning physical product. We integrated the smart contracts with a robust backend, built the automated 5-stage verification pipeline, and implemented an Urdu notification module. Most importantly, we bridged the gap between software and the real world by successfully integrating physical plastic cards and RFID readers for point-of-sale transactions.

Our testing confirms that the system meets its performance goals and maintains perfect data integrity between the physical hardware scans, the database, and the blockchain. The modular design means that future teams or organizations could easily swap our simulated NADRA datasets for live government APIs and deploy this system to a national production environment.

The primary limitation of the current prototype is its scope: real NADRA and SBP integrations, industrial-grade card security, and large-scale multi-institution deployment are explicitly out of scope. Future extensions could address these by integrating with live government APIs, adopting EMV-compliant card technology, and scaling the blockchain network to a multi-organization Hyperledger Fabric deployment. The modular GRC framework design ensures that these extensions can be incorporated without redesigning the core eligibility and policy enforcement logic.

Overall, AidLedger is a solid, engineering-focused demonstration of how decentralized technology combined with hardware can create a fairer and highly transparent public welfare system - an increasingly relevant contribution as Pakistan and similar economies explores digital transformation of their social protection systems.

References

[1] World Food Programme. "Blockchain against hunger: Harnessing technology in support of Syrian refugees," WFP Report, 2019.

[2] S. Ølnes, J. Ubacht, and M. Janssen, "Blockchain in government: Benefits and implications," Government Information Quarterly, vol. 34, no. 3, pp. 355–364, 2017.

[3] A. Khan, "Challenges in BISP: Need for transparency and reforms," Journal of Public Policy, vol. 12, no. 2, pp. 45–53, 2020.

[4] Deloitte. "RegTech in financial services: GRC on blockchain," Deloitte Insights, 2021.

[5] A. Tapscott and D. Tapscott, Blockchain Revolution: How the Technology Behind Bitcoin Is Changing Money, Business, and the World, Penguin, 2018.

[6] N. Kshetri, "Blockchain's roles in strengthening cybersecurity and protecting privacy," Telecommunications Policy, vol. 41, no. 10, 2017.

[7] H. Treiblmaier, "The impact of the blockchain on the supply chain: A theory-based research framework and a call for action," Supply Chain Management Journal, vol. 23, no. 6, 2018.

[8] State Bank of Pakistan, "Regulations on AML/CFT and KYC," SBP Circular, 2023.

[9] UNDP Pakistan, "Ehsaas and BISP program evaluation report," UNDP, 2022.

[10] "Rs141b BISP fraud exposed in audit," The Express Tribune, Mar. 19, 2025.